

UNIVERSITE DE SOUSSE Institut Supérieur des Sciences Appliquées et de Technologie de Sousse Sousse	Examen TP	Classe : ING-A2-GL A.U. : 2025/2026
--	------------------	--

Dans le cadre d'un système de gestion financière, une entreprise souhaite automatiser le suivi des factures clients ainsi que les paiements associés. Chaque client peut avoir plusieurs factures, et chaque facture peut être réglée partiellement ou totalement à travers un ou plusieurs paiements.

Vous êtes amené(e) à manipuler cette base en utilisant des requêtes SQL avancées ainsi que des programmes PL/SQL (fonctions, triggers, packages).

Partie 1 : Création de la base de données

Exécuter le script suivant pour créer et initialiser la base de données :

```

DROP TABLE paiements CASCADE CONSTRAINTS;

DROP TABLE factures CASCADE CONSTRAINTS;

DROP TABLE clients CASCADE CONSTRAINTS;


CREATE TABLE clients (

    id_client NUMBER PRIMARY KEY,

    nom VARCHAR2(50),

    type_client VARCHAR2(20)

);


CREATE TABLE factures (

    id_facture NUMBER PRIMARY KEY,

    id_client NUMBER REFERENCES clients(id_client),

```

```
montant NUMBER(10,2),

date_facture DATE,

statut VARCHAR2(20)

);


CREATE TABLE paiements (

    id_paiement NUMBER PRIMARY KEY,

    id_facture NUMBER REFERENCES factures(id_facture),

    montant_paye NUMBER(10,2),

    date_paiement DATE

);
```

Partie 2 : Insertion des données

```
INSERT INTO clients VALUES (1, 'Ali', 'VIP');
INSERT INTO clients VALUES (2, 'Sami', 'NORMAL');

INSERT INTO factures VALUES (1, 1, 2000, SYSDATE, 'NON_PAYEE');
INSERT INTO factures VALUES (2, 1, 3000, SYSDATE, 'NON_PAYEE');
INSERT INTO factures VALUES (3, 2, 1500, SYSDATE, 'NON_PAYEE');

INSERT INTO paiements VALUES (1, 1, 1000, SYSDATE);
INSERT INTO paiements VALUES (2, 3, 1500, SYSDATE);

COMMIT;
```

Travail demandé

Question 1 : Écrire une requête SQL permettant d'afficher, pour chaque client, les informations suivantes : **id_client** | **total_facture** | **total_paye** | **reste_a_payer** | **statut_client**

Avec :

- **total_facture** : somme des montants de toutes les factures du client
- **total_paye** : somme de tous les paiements effectués par le client
- **reste_a_payer** = **total_facture** - **total_paye**
 - ✓ **statut_client** défini comme suit : **BON_PAYEUR** si le reste à payer est égal à 0
 - ✓ **CLIENT_A_RISQUE** si le reste à payer est supérieur à la moyenne des restes de tous les clients
 - ✓ **INTERMEDIAIRE** sinon

Contraintes :

- utiliser obligatoirement GROUP BY
- utiliser CASE
- utiliser une sous-requête pour calculer la moyenne globale

Question 2 : Créer une fonction : **FUNCTION Total_Reste_Client(p_id_client NUMBER) RETURN NUMBER;**

Cette fonction doit :

- calculer et retourner le reste à payer d'un client donné
- retourner 0 si le client n'a aucune facture
- lever une exception si le client n'existe pas : **RAISE_APPLICATION_ERROR(-20002, 'Client inexistant');**

Question 3 : Créer un trigger **BEFORE INSERT** sur la table paiements.

Ce trigger doit :

1. Vérifier que le montant du paiement ne dépasse pas le reste à payer de la facture concernée.
2. Refuser l'insertion avec un message d'erreur explicite si la condition n'est pas respectée.
3. Si l'insertion est valide : mettre à jour automatiquement le statut de la facture :

PAYEE si le montant total payé atteint le montant de la facture,

PARTIELLE sinon.